Data Typing and Structuring Using Python

Explore how to apply data typing and structuring using Python to standardize data formats, organize data into suitable structures, and merge data sources. This lesson helps you understand key techniques to prepare extracted data for effective loading and analysis in ETL pipelines.

Data typing and data structuring often go hand in hand and are some of the first transformations we need to do. In ETL pipelines, the destination schema or repository will heavily dictate the type of typing and structuring we need.

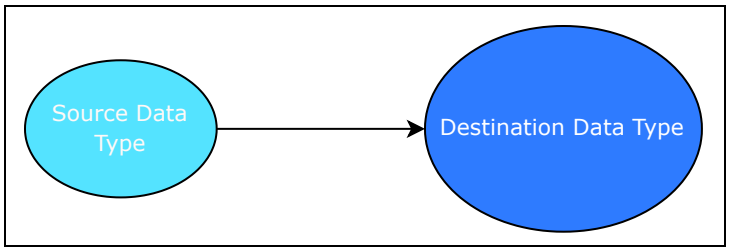
Data typing

Data typing involves converting a column or multiple columns of data to a standardized data type such as integer, float, string, or boolean or changing how the data is represented.

An ETL pipeline usually extracts data from many different sources, and the extracted data is expected to have some inconsistencies. For example, we often need to convert date formats from different sources, like DD/MM/YYYY or MM/DD/YYYY, to a common format, like YYYY-MM-DD, to enable easier comparison, processing, and loading of the data. This is an example of using data typing and changing a column of data from one string representation (DD/MM/YYYY) to another (MM/DD/YYYY).

Ask

Data typing helps ensure that the transformed data is consistent and is critical for meeting the data type constraints of the destination schema.



Casting data types

Furthermore, by standardizing the data type, we can rely on built-in functions that accept only a certain data type. For example, if a field is defined as an integer, we can run mathematical functions like `max`, `min`, `average`, and `sum` on the entire column of integers.

Data structuring

Data structuring refers to the process of organizing data in a specific way in order to make it easier to manage and analyze.

This can involve creating new columns or dividing data into smaller, manageable units, such as arrays, lists, queues, or stacks. It can also involve creating relationships between data elements, such as parent-child relationships in a tree structure or between tables in a relational database. A common example of data structuring in ETL pipelines is to convert data from one format, such as JSON, XML, or CSV, into database tables.

Ask

Converting JSON to a table

Data structuring aims to make data more organized, efficient, and accessible for processing and analysis.

Example

Suppose we're developers working for an online retailer. We need to extract the sales data from each department, merge it, and analyze it. The data is stored in different CSV files, each representing a sales month. The problem is that the departments store the CSV files in a slightly different structure, with different column names and date formats.

We need to transform the data into a common format to make it usable for analysis. Currently, there are two departments. The CSV file for January for the first department looks like this:

Ask

department1_jan.... Shell

department1_jan.csv

The CSV file for the second department looks like this:

department2_jan.... C++

department2_jan.csv

According to the business requirements, there are a few issues that we ne Ask fix:

- The date format must be DD-MM-YYYY.

- The column names must be `Date`, `Product`, `Revenue`, and `Cost`.
- The column `Cost` must be of integer type.
- Finally, we need to merge the data from all departments into a single CSV file.

Given these requirements, we need to transform the data to conform to the correct type and structure. The following Python function will fix these issues. The function takes the `number_of_departments` and the `current_month` as arguments.

We start by converting each department's CSV file into DataFrame and append all DataFrames into a single list. Then, we iterate over the list of DataFrames and transform each one according to the specified requirements.

In the end, we merge the DataFrames into a single, large DataFrame and export it as CSV. Click the "Run" button in the following code to apply data typing and structuring to the data:

main.py Python 3.10.4

```
import pandas as pd
import datetime

def merge_departments_csv(number_of_departments, current_month):

    # Extracting CSV files into a list of DataFrames
    print('Extracting CSV files into a list of DataFrames')
    dfs = [pd.read_csv(f'/usercode/tmp/department{i}_{current_month}.csv'

    # Expected columns
    expected_columns = ['date', 'product', 'revenue', 'cost']

    # For each DataFrame: rename columns, format date and change data typ
```

Ask

Transforming sales department's CSV files

- **Line 8:** We use a for loop to read the CSV files into a list of DataFrames.
- **Line 21:** We rename old columns into new ones for each DataFrame.
- **Lines 24–29:** We change the date format for each department to DD-MM-YYYY. We use `dayfirst=True` to give Python information about the current structure of the date format for each department.

Ask